Blatt 1

Kryptologie

Luis Staudt

Analyse eines LFSR-PRNG

Aufgabenstellung

Es soll ein Linear Feedback Shift Register (LFSR) vom Fibonacci-Typ der Länge 16 analysiert werden. Das Generatorpolynom lautet:

$$g(x) = x^{16} + x^{10} + x^{6} + x^{5} + 1$$

Der Initialwert (als Bitfolge) ist:

$$(1,\ 0,\ 0,\ 1,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1)$$

Es werden 20000 Bits erzeugt und anschließend mit dem Monobit-Test, dem Poker-Test und dem Run-Test auf statistische Zufälligkeit geprüft. Die Annahmebereiche richten sich nach NIST FIPS140-2 und BSI AIS 31.

Die ersten und letzten 25 erzeugten Bits lauten:

$$\begin{aligned} \text{Erste 25 Bits:} &\quad 1\,1\,1\,1\,1\,1\,0\,0\,0\,0\,0\,0\,1\,0\,0\,1\,1\,0\,0\,1\,0\,1\,0\,0\,1 \\ \text{Letzte 25 Bits:} &\quad 1\,0\,0\,1\,0\,0\,0\,1\,1\,0\,0\,0\,1\,0\,1\,1\,1\,0\,0\,0\,0\,0\,0\,1\,0 \end{aligned}$$

Vorgehensweise

Die Implementierung erfolgt in Python. Das LFSR wird als Array der Länge 16 realisiert. In jedem Takt wird das Ausgabebit (Position 16) entnommen und das Feedback als XOR der Positionen 16, 10, 6 und 5 berechnet. Anschließend wird das Register nach rechts geschoben und das Feedback-Bit an Position 1 eingefügt.

```python
lfsr = [1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
bits = []
for i in range(20000):
    output = lfsr[15]
    bits.append(output)
    fb = (lfsr[15] + lfsr[9] + lfsr[5] + lfsr[4]) % 2
    lfsr = [fb] + lfsr[:15]
```

Code Listing 1: LFSR-Implementierung

Monobit-Test

Der Monobit-Test zählt die Anzahl der Einsen X in der gesamten Bitsequenz. Bei einer idealen Zufallsfolge erwartet man $X \approx 10000$.

Test bestanden gemäß	Wert	Annahmebereich	Ergebnis
NIST FIPS140-2	$X = 9972$	$(9725;\ 10275)$	bestanden
BSI AIS 31		$(9654;\ 10346)$	bestanden

Tabelle 1: Ergebnis des Monobit-Tests

Der Wert $X = 9972$ liegt nahe am Erwartungswert und innerhalb beider Annahmebereiche. Der Monobit-Test ist somit bestanden.

Poker-Test

Beim Poker-Test werden die 20000 Bits in 5000 Nibbles (je 4 Bit) aufgeteilt. Die Zufallsgröße f_k zählt die Häufigkeit jedes Nibble-Werts $k \in \{0, \ldots, 15\}$. Die Teststatistik berechnet sich als:

$$Y = \frac{16}{5000} \cdot \sum_{k=0}^{15} f_k^2 - 5000$$

Test bestanden gemäß	Wert	Annahmebereich	Ergebnis
NIST FIPS140-2	$Y = 14{,}7968$	$(2{,}16;\ 46{,}17)$	bestanden
BSI AIS 31		$(1{,}03;\ 57{,}4)$	bestanden

Tabelle 2: Ergebnis des Poker-Tests

Die Häufigkeitsverteilung der Nibble-Werte ist in figure 1 dargestellt.

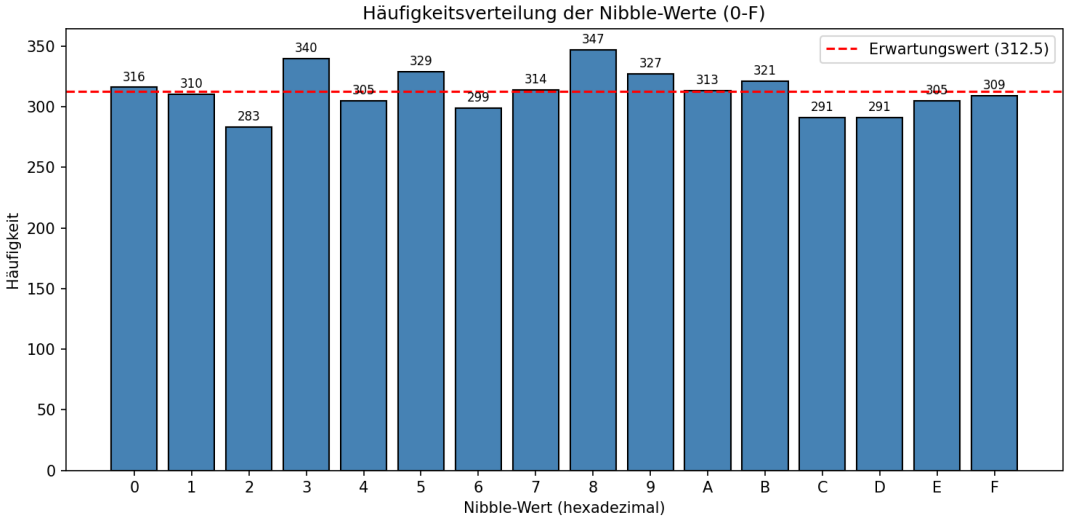

Abbildung 1: Häufigkeitsverteilung der Nibble-Werte (0–F)

Die Nibble-Häufigkeiten schwanken um den Erwartungswert von $312{,}5$ und zeigen keine auffällige Abweichung. Der Poker-Test ist bestanden.

Run-Test

Der Run-Test untersucht die Anzahl der aufeinanderfolgenden gleichen Bits (Runs). Es wird zwischen 0-Runs und 1-Runs unterschieden, jeweils nach Länge k gruppiert.

Länge	$zero_k$	one_k	**NIST Bereich**	**NIST**	**BSI Bereich**	**BSI**
1	2476	2534	$[2315;\ 2685]$	best.	$[2267;\ 2733]$	best.
2	1263	1207	$[1114;\ 1386]$	best.	$[1079;\ 1421]$	best.
3	631	621	$[527;\ 723]$	best.	$[502;\ 748]$	best.
4	301	329	$[240;\ 384]$	best.	$[233;\ 402]$	best.
5	163	134	$[103;\ 209]$	best.	$[90;\ 223]$	best.
≥ 6	159	168	$[103;\ 209]$	best.	$[90;\ 223]$	best.

Tabelle 3: Ergebnis des Run-Tests

Die Verteilung der Runs ist in figure 2 dargestellt.

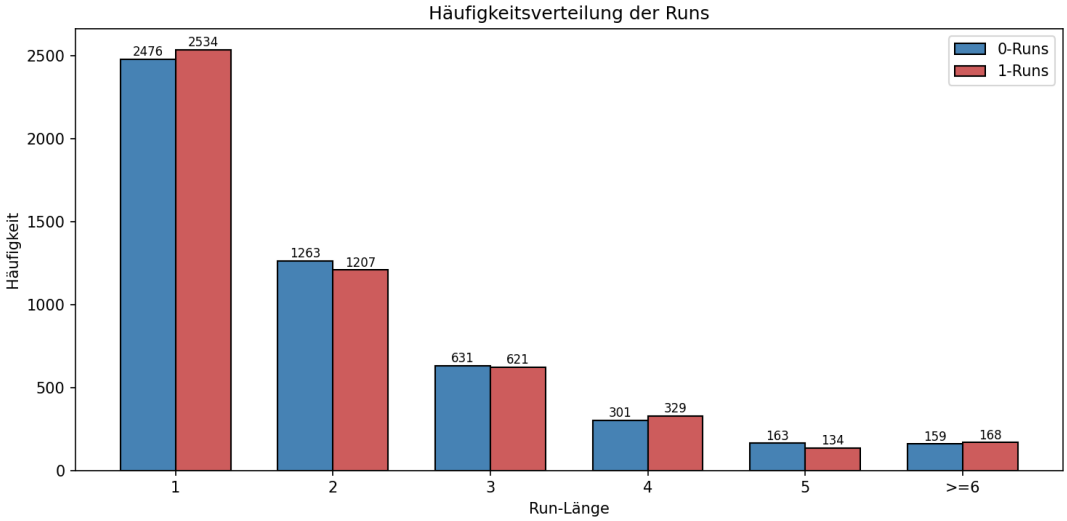

Abbildung 2: Häufigkeitsverteilung der 0-Runs und 1-Runs

Alle Run-Häufigkeiten liegen innerhalb der Annahmebereiche beider Standards. Der Run-Test ist bestanden.

Long-Run-Test

Der Long-Run-Test prüft, ob ungewöhnlich lange Runs auftreten.

Test bestanden gemäß	Längster Run	Grenzwert	Ergebnis
NIST FIPS140-2	13	< 34	bestanden
BSI AIS 31		< 31	bestanden

Tabelle 4: Ergebnis des Long-Run-Tests

Der längste aufgetretene Run hat die Länge 13 und liegt weit unter beiden Grenzwerten.

Bewertung

Das LFSR mit dem Generatorpolynom $g(x) = x^{16} + x^{10} + x^6 + x^5 + 1$ besteht alle vier statistischen Zufälligkeitstests (Monobit, Poker, Run, Long-Run) sowohl nach NIST FIPS140-2 als auch nach BSI AIS 31. Die erzeugten 20000 Bits weisen eine gleichmäßige Verteilung der Einsen und Nullen auf, die Nibble-Häufigkeiten schwanken nahe am Erwartungswert und die Run-Längen liegen in den erwarteten Bereichen.

Das Generatorpolynom erzeugt somit eine Bitfolge, die grundlegende statistische Zufälligkeitsanforderungen erfüllt. Es ist jedoch zu beachten, dass diese Tests notwendige, aber nicht hinreichende Bedingungen für kryptographische Sicherheit sind. Ein LFSR allein ist als kryptographischer Zufallsgenerator nicht geeignet, da die lineare Struktur durch den Berlekamp-Massey-Algorithmus leicht rekonstruiert werden kann.